

Manual do Projeto

O que cada arquivo executa, organizado exatamente como a árvore de pastas do repositório. Nome do arquivo linka direto pro código no GitHub. Comandos de execução e teste no final.

ÍNDICE

01 Raiz do projeto	05 app/infrastructure/
02 app/	06 migrations/
03 app/domain/	07 tests/
04 app/application/	08 Comandos

COMO USAR

Cada seção abaixo é uma pasta real do projeto. Dentro dela, cada linha é um arquivo com uma frase dizendo o que ele executa — não o que ele é. Clique no nome do arquivo pra abrir o código correspondente no GitHub.

/ (raiz)

Configuração de ambiente, orquestração e documentação — nada de regra de negócio aqui.

README.md

Documentação principal: arquitetura, decisões técnicas, como rodar e como testar.

Dockerfile

Builda a imagem usada pelos serviços `app` e `consumer` no compose.

docker-compose.yml

Orquestra mysql, redis, kafka, app e consumer; roda `alembic upgrade head` antes de subir a API.

requirements.txt

Lista as dependências Python fixadas: FastAPI, SQLAlchemy, kafka-python, redis, pytest e afins.

pytest.ini

Define `testpaths`, cobertura mínima (70%) e os markers `unit` / `integration`.

alembic.ini

Aponta o Alembic pra pasta `migrations/` e pro banco definido em `DATABASE_URL`.

.env.example

Modelo das variáveis de ambiente: URLs de MySQL/Kafka/Redis, tópicos, TTL de cache.

consumer.py

Processo worker: consome `requests.created`, aplica a regra de negócio, trata falha com retry e DLQ.

architecture.pdf / .drawio

Diagrama do fluxo ponta a ponta (o `.pdf` é o exportado pra leitura; o `.drawio` é o fonte editável).

architecture-layers.pdf / .drawio

Diagrama das camadas — domínio, aplicação, infraestrutura (hexagonal).

.github/workflows/ci.yml

Pipeline de CI: roda os testes com cobertura, depois builda a imagem Docker (só se os testes passarem).

app/

Ponto de entrada da aplicação e a composição de tudo que ela depende.

main.py

Cria a instância FastAPI, registra o middleware de correlation ID, inclui as rotas, expõe `/health` e `/health/ready`, e o handler genérico de erro 500.

composition.py

Composition root: cria a engine do MySQL e o cliente Redis, e monta as factories que o FastAPI injeta via `Depends` — liga cada use case ao seu repositório/publisher/cache reais.

app/domain/

Regra de negócio pura. Nada aqui importa FastAPI, SQLAlchemy, Kafka ou Redis.

entities.py

Define a entidade `Request` (dataclass): id, customer_id, value, status, timestamps.

enums.py

Define `RequestStatus`: PENDING, APPROVED, MANUAL_REVIEW, FAILED.

services.py

Executa `decide_status()` — a regra de negócio pura (≤ 1000 aprova, > 1000 vai pra revisão manual).

app/application/

Orquestra o domínio contra interfaces (portas) — nunca contra tecnologia concreta.

ports.py

Define os contratos (`Protocol`): `RequestRepository`, `EventPublisher`, `CachePort`.

use_cases/create_request.py

Executa `CreateRequestUseCase`: cria a solicitação, persiste, publica o evento no Kafka.

use_cases/get_request.py

Executa `GetRequestUseCase`: consulta com cache-aside — Redis, miss vai no MySQL, popula o cache.

use_cases/process_request.py

Executa `ProcessRequestUseCase`: aplica `decide_status`, atualiza o status, invalida o cache — contém a guarda de idempotência.

app/infrastructure/

Implementa as portas com tecnologia real e hospeda os adapters de entrada (HTTP e Kafka).

config.py

Define `Settings` (pydantic-settings): variáveis de ambiente tipadas — `database_url` obrigatório, sem default.

logging.py

Configura o formatter de log em JSON e o `ContextVar` que carrega o `correlation_id`.

http/router.py

Define as rotas `POST /requests` e `GET /requests/{id}`.

http/schemas.py

Define os schemas Pydantic de entrada/saída e as validações (`value > 0`, `customer_id` não vazio).

http/middleware.py

Executa `CorrelationIdMiddleware`: gera ou propaga o header `X-Correlation-Id`.

http/rate_limit.py

Executa `check_rate_limit`: dependency do FastAPI, faz INCR+EXPIRE atômico no Redis.

persistence/models.py

Define o modelo SQLAlchemy da tabela `requests`.

persistence/repository.py

Executa `SQLAlchemyRequestRepository` — implementa `RequestRepository` contra o MySQL.

messaging/producer.py

Executa `KafkaEventPublisher` — implementa `EventPublisher`, publica no Kafka.

cache/redis_cache.py

Executa `RedisCache` — implementa `CachePort` sobre o cliente Redis.

cache/keys.py

Executa `request_cache_key()` — monta a chave `request:{id}` usada em todo lugar que toca o cache.

migrations/

Alembic — versionamento do schema do MySQL.

env.py

Configura o ambiente do Alembic — de onde ele lê `DATABASE_URL` e os metadados dos models.

script.py.mako

Template usado quando uma nova migration é gerada (`alembic revision`).

versions/0001 create requests.py

Executa a criação da tabela `requests` — a única migration do projeto até agora.

tests/

Suíte completa — não depende de Docker, MySQL, Kafka ou Redis reais.

conftest.py

Fixture `sqlite_session` — banco SQLite em memória compartilhado entre conexões via `StaticPool` .

fakes.py

`FakeRepo` , `FakePublisher` , `FakeCache` — duplões em memória usados nos testes unitários.

unit/test_domain.py

Testa `decide_status` isoladamente.

unit/test_use_cases.py

Testa os 3 use cases contra os fakes, sem infraestrutura real.

unit/test_rate_limit.py

Testa `check_rate_limit` com um Redis mockado — garante o uso do pipeline transacional.

unit/test_consumer.py

Testa o laço do consumer com Kafka e banco mockados — commit de offset, DLQ, retry.

[integration/test_repository.py](#)

Testa `SQLAlchemyRequestRepository` contra SQLite real.

[integration/test_e2e.py](#)

Testa a API via `TestClient` fim a fim — POST/GET, validação, 404.

[integration/test_consumer_flow.py](#)

Testa o consumer real (não mockado) contra SQLite + dublês de Kafka/Redis.

Comandos

O que cada comando executa, na ordem em que você provavelmente vai usá-los.

Sem Docker — só os testes automatizados

```
$ python -m pip install -r requirements.txt
```

Instala as dependências do `requirements.txt` no Python ativo (3.10+). Usa `python -m pip` em vez de só `pip` porque, no Windows, a pasta `Scripts` (onde fica `pip.exe`) nem sempre está no PATH — passando pelo `python`, isso deixa de importar.

```
$ python -m pytest
```

Roda toda a suíte (unitários + integração) com o relatório de cobertura definido em `pytest.ini`. Usa SQLite em memória e os dublês de `tests/fakes.py` — não depende de MySQL, Kafka ou Redis reais.

```
$ python -m pytest -m unit
```

Roda só os testes marcados `unit` — domínio e use cases, sem nenhuma infraestrutura simulada.

```
$ python -m pytest -m integration
```

Roda só os testes marcados `integration` — repositório contra SQLite real, API via `TestClient`, consumer contra fakes.

```
$ python -m pytest --no-cov
```

Roda a suíte sem calcular cobertura — mais rápido pra iteração local. Precisa ser `--no-cov`, não `-p no:cov`: como o `pytest.ini` já injeta `--cov=app` via `addopts`, desativar o plugin com `-p no:cov` deixa esse argumento órfão e o pytest recusa a rodar.

Com Docker — stack completo

```
$ cp .env.example .env
```

Copia o modelo de variáveis de ambiente pro arquivo que o compose e a aplicação de fato leem.

```
$ docker compose up --build
```

Builda a imagem e sobe mysql, redis e kafka; assim que cada um fica saudável, sobe `app` (que roda `alembic upgrade head` e depois `uvicorn`) e `consumer` (`python consumer.py`). Kafka pode levar ~2 min pra ficar healthy — é normal ver `app` / `consumer` reiniciando até lá.

```
$ docker compose ps
```

Mostra o status de cada serviço — confirma que todos chegaram em `healthy` / `Up` antes de testar.

```
$ docker compose logs -f consumer
```

Acompanha em tempo real o consumer processando mensagens — útil pra mostrar o processamento assíncrono acontecendo.

```
$ docker compose down
```

Para e remove os containers da stack ao final.

Verificando e exercitando a API (PowerShell)

No PowerShell, `curl` é apelido de `Invoke-WebRequest` — não aceita `-H/-d` como o `curl` de verdade e ainda pede confirmação de segurança antes de exibir a resposta. Os comandos abaixo usam `Invoke-RestMethod`, que evita os dois problemas.

```
$ Invoke-RestMethod http://localhost:8000/health/ready
```

Confirma que a API está de pé e que MySQL e Redis estão acessíveis a partir dela. Retorna `status: ready`.

```
$ $body = @{ customer_id = "123"; value = 1500.00 } | ConvertTo-Json
$created = Invoke-RestMethod -Uri http://localhost:8000/requests -Method
Post -Body $body -ContentType "application/json"
$created
```

Cria uma solicitação — persiste **PENDING** no MySQL e publica o evento no Kafka. **\$created** guarda a resposta inteira, **201** com o ID gerado em **\$created.id**.

```
$ Start-Sleep -Seconds 2
Invoke-RestMethod "http://localhost:8000/requests/$( $created.id )"
```

Consulta a mesma solicitação usando o ID que já está em **\$created.id** — sem precisar copiar/colar UUID na mão. Chamado logo após o POST ainda pode mostrar **PENDING**; ~2s depois já mostra **MANUAL_REVIEW** ou **APPROVED**, porque o consumer processou entre as duas chamadas.

```
$ start http://localhost:8000/docs
```

Abre o Swagger gerado pelo FastAPI no navegador padrão — interface visual pra testar os dois endpoints sem escrever comando nenhum. Colar a URL direto no PowerShell não funciona: ele tenta rodar a URL como comando, não abrir no navegador.

Referência de navegação do repositório — não substitui o README, só facilita achar onde cada coisa está.